

ISM3232 — Week 11 Lab

OOP II — Composition, Inheritance & SQL Mapping — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 11 · Unit 3 · Object-Oriented Design

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	5
6.1 Intro (0:00–0:04)	5
6.2 Part 1 — Write RequestManager (0:04–0:20, 16 min)	5
6.3 Part 2 — Wire the Two Classes (0:20–0:30, 10 min)	8
6.4 Part 3 — OOP to SQL Mapping (0:30–0:42, 12 min)	10
6.5 Part 4 — Six pytest Tests (0:42–0:58, 16 min)	12
6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)	15
6.7 Stretch — Light Inheritance (1:06–1:15, as time allows)	16
7 Wrap-Up (last ~9 minutes)	17
8 Appendix A — Full Answer Key (models.py additions + main.py + README.md section + tests/test_models.py additions)	17
9 Appendix B — Extra Practice (only if the class finishes early)	20

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 11 Lab — OOP II: Composition, Inheritance & SQL Mapping
Unit	Unit 3 · Object-Oriented Design
Class length	Full class period (75 minutes)
Format	Live code-along, ending with the full submission ritual
Prerequisites	Week 10: BusinessRequest class, <code>__init__</code> , <code>self</code> , instance methods, <code>__repr__</code>
Student-facing lab page	Week 11 In-Class Lab — Module 6, “OOP II: Composition and the Manager Class”
Parts covered	Part 1 (RequestManager) – Part 5 (ritual + push) + Stretch (light inheritance)
Submission	2 screenshots + README.md with OOP-to-SQL mapping, GitHub URL, Canvas, completion credit

Week 10 built one class, standing alone. Today introduces **composition** — a RequestManager class whose entire job is to *hold and manage a collection* of BusinessRequest objects, not replace them. This is a genuinely different relationship between classes than the inheritance shown briefly in the Stretch section, and the distinction is worth stating explicitly: composition is “has-a” (a manager *has* a list of requests), inheritance is “is-a” (a TravelRequest *is* a kind of BusinessRequest). Part 3’s OOP-to-SQL mapping table is also worth taking seriously as more than a formality — it’s this course’s first explicit bridge toward Week 13–14’s database content, and getting students to genuinely map class → table, instance → row, attribute → column now pays off directly in three weeks.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Explain composition — one class holding instances of another as an attribute — and distinguish it from inheritance.
2. Write manager-style methods that filter, aggregate, and report on a collection of objects using list comprehensions and generator expressions over object attributes.
3. Map basic OOP concepts (class, instance, attribute, method) to their SQL equivalents (table, row, column, query).

4. Write tests that verify a composed system — a manager holding multiple, independently-tracked objects — including a test that two manager instances are themselves independent.
5. (Stretch) Write a subclass using `super().__init__(...)` and override an inherited method.

3 Before Class — Setup Checklist

- ☐ Rehearse the composition-vs-inheritance distinction explicitly before class — today's core content (RequestManager holding BusinessRequests) is composition; the Stretch (TravelRequest(BusinessRequest)) is inheritance — having a clear, ready one-sentence contrast for each avoids the two ideas blurring together if a student asks about both in the same breath.
- ☐ Fill in your own OOP-to-SQL mapping table (Part 3) with genuine, specific SQL before class — a real `CREATE TABLE requests (...)` statement and a real `SELECT * FROM requests WHERE status = 'Pending'` query, ready to show as a model, make the abstract mapping concrete in a way the bare table template doesn't on its own.
- ☐ Confirm Week 10's `models.py` and `tests/test_models.py` are correctly carried forward into this week's project folder (same `module06_oop/` directory, not a fresh one) — today's Part 1 explicitly adds to the bottom of the existing file, and Part 4's tests explicitly say “keep your Week 10 tests.”

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module06_oop/` venv and files from Week 10
- Students: their Week 10 `models.py`, `main.py`, and `tests/test_models.py`, continuing in place

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: “one class managing many of another”	4
0:04–0:20	Part 1 — Write RequestManager	16
0:20–0:30	Part 2 — Wire the two classes	10
0:30–0:42	Part 3 — OOP to SQL mapping	12
0:42–0:58	Part 4 — Six pytest tests	16
0:58–1:06	Part 5 — Ritual and push	8
1:06–1:15	Stretch (light inheritance) + wrap-up	9

Part 1 and Part 4 receive the most time, for the same reason as Week 10: new conceptual ground

(composition, and testing a *composed* system) deserves unhurried treatment; Part 3's SQL mapping is comparatively quick to execute but worth framing as genuinely important groundwork, not busywork, given its direct payoff in Weeks 13–14.

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Last week, one `BusinessRequest` stood alone. Today, a second class — `RequestManager` — whose entire job is holding a collection of `BusinessRequest` objects and answering questions about them: how many are pending, what’s the total value, give me a report. This relationship — one class containing instances of another as an attribute — is called composition, and it’s genuinely different from the inheritance you’ll see briefly in today’s Stretch section. Composition: a manager has requests. Inheritance, coming later: a travel request is a request. Both matter; they’re not the same idea.”

6.2 Part 1 — Write `RequestManager` (0:04–0:20, 16 min)

Teaching goal: A class whose primary attribute is a **list of other objects**, plus methods that filter and aggregate over that list using patterns from every prior week (list comprehensions, generator expressions) now applied to objects instead of dictionaries.

Say to the class:

“Five methods. Watch specifically how much of this is genuinely new syntax versus patterns you already know from Weeks 6 and 7, just applied to `BusinessRequest` objects instead of dictionaries.”

Live-code this, added to the bottom of `models.py`:

```
class RequestManager:
    """Manages a collection of business requests."""

    def __init__(self):
        self.requests = []

    def add_request(self, request):
        """Add a BusinessRequest to the collection."""
        self.requests.append(request)

    def list_pending(self):
        """Return all requests with status Pending."""
        return [r for r in self.requests if r.status == 'Pending']

    def total_amount(self):
```

```

        """Return the total value of all requests."""
        return sum(r.amount for r in self.requests)

    def get_by_status(self, status):
        """Return requests matching the given status."""
        return [r for r in self.requests if r.status == status]

    def summary_report(self):
        """Print a formatted summary of all requests by status."""
        pending = self.list_pending()
        approved = self.get_by_status('Approved')
        print(f'Total requests: {len(self.requests)} | Total: ${self.total_amount():,.2f}')
        print(f'Pending: {len(pending)} | Approved: {len(approved)}')
        print('Pending requests:')
        for r in pending:
            print(f' {r}')

```

Line-by-line explanation:

- `def __init__(self): self.requests = []` — **this is composition, in one line.** `self.requests` is an attribute, exactly like Week 10's `self.status` — but instead of a string, it holds a **list**, initially empty, that will come to hold actual `BusinessRequest` **objects**. Say explicitly: a `RequestManager` doesn't inherit anything from `BusinessRequest`, doesn't share any code with it — it simply *has* a list that will contain instances of it. This is the entire mechanism of composition: one object holding references to other objects as its own attribute.
- `def add_request(self, request):` — takes an *already-created* `BusinessRequest` object as a parameter (named `request`, distinct from `self`) and appends it to `self.requests` — say explicitly, this method doesn't create a new request itself; it receives one that already exists (Part 2 shows exactly how) and adds it to the collection.
- `list_pending`, `get_by_status` — both list comprehensions filtering `self.requests` — **note `r.status`, not `r['status']`** — say explicitly, this is the direct, concrete difference from Week 6's dictionary-based filtering: these are real objects with **attributes**, accessed with dot notation, not dictionaries with keys accessed by brackets. The *shape* of the comprehension (`[r for r in collection if condition]`) is identical to Week 6; only the access syntax for reading each item's data has changed.
- `total_amount` — `sum(r.amount for r in self.requests)` — a generator expression, exactly Week 6's `test_accumulator` pattern, now summing an attribute across a collection of objects instead of plain numbers in a list.
- `summary_report` — calls `self.list_pending()` and `self.get_by_status('Approved')` — **note these are the manager calling its own other methods**, via `self`, exactly the same mechanism as any external code calling `mgr.list_pending()` — worth flagging explicitly as a small but genuinely useful pattern: methods can call other methods on the same object,

reusing logic rather than duplicating it.

- `print(f' {r}')` inside the final loop — `r` is a `BusinessRequest` object; printed inside an f-string, it uses Week 10's `__repr__` automatically — say explicitly, this is the same mechanism from last week's `print(req_101)`, now happening implicitly, inside a loop, inside a different class's method.

Common student mistakes to watch for:

- Using `r['status']` (dictionary bracket access) out of Week 6 habit instead of `r.status` (attribute access) — raises `TypeError: 'BusinessRequest' object is not subscriptable`, a good, specific error worth reading together, since it directly names the conceptual mix-up (treating an object like a dictionary).
- Forgetting `self.requests = []` in `__init__`, or initializing it to something other than an empty list — every method that follows assumes `self.requests` is a list that can be appended to and iterated over; a wrong initial type breaks everything downstream in a way that's easy to trace back to this one line.

Check for understanding: “If `RequestManager` had a second attribute, `self.name`, set from a parameter in `__init__`, would that change anything about how `add_request` or `list_pending` work?” (No — those methods only ever reference `self.requests`; an unrelated attribute like `self.name` would simply coexist on the same object, untouched by these methods. A good check that students see `self.requests` as *the specific attribute these methods care about*, not “everything about the object.”)

6.3 Part 2 — Wire the Two Classes (0:20–0:30, 10 min)

Teaching goal: Create a manager, populate it with several `BusinessRequest` objects, and confirm the two classes work together correctly — composition made concrete.

Say to the class:

“Now the two classes meet. A manager, four requests added to it one at a time, one approved, then a full report.”

Live-code this, replacing `main.py`’s Week 10 content:

```
# main.py
from models import BusinessRequest, RequestManager

mgr = RequestManager()
mgr.add_request(BusinessRequest(101, 'Taylor', 'Travel', 1200))
mgr.add_request(BusinessRequest(102, 'Jordan', 'Equipment', 450))
mgr.add_request(BusinessRequest(103, 'Morgan', 'Software', 3500))
mgr.add_request(BusinessRequest(104, 'Riley', 'Travel', 89))

# Approve one request
mgr.requests[0].approve()

# Print the full report
mgr.summary_report()
```

Line-by-line explanation:

- `from models import BusinessRequest, RequestManager` — importing **two** names from the same module now, both classes, side by side — worth noting the import line itself is a visible signal of the two-class design.
- `mgr = RequestManager()` — creates the (initially empty) manager — no arguments, since `__init__(self)` takes none beyond `self`.
- `mgr.add_request(BusinessRequest(101, 'Taylor', 'Travel', 1200))` — **read this inside-out, since it’s doing two things in one line:** `BusinessRequest(101, 'Taylor', 'Travel', 1200)` creates a brand-new object first; that object, not yet stored in any variable of its own, is immediately passed into `mgr.add_request(...)`, which appends it to `mgr.requests`. Say explicitly: this is valid and common — an object doesn’t need its own named variable to be used immediately as an argument to something else.
- `mgr.requests[0].approve()` — **note the double access:** `mgr.requests` gets the list; `[0]` indexes into it (Module 10’s Taylor request, the first one added); `.approve()` then calls a method on *that specific* `BusinessRequest` object. Say explicitly: this line is reaching *through* the manager, into its held collection, to act on one specific request directly — the

manager itself doesn't need an `approve_first()` method or similar for this to work, since `mgr.requests[0]` hands back the actual object, with all its own methods intact.

- `mgr.summary_report()` — calls the report method built in Part 1.

Run it. Verified output:

```
Total requests: 4 | Total: $5,239.00
```

```
Pending: 3 | Approved: 1
```

```
Pending requests:
```

```
    BusinessRequest(102, Jordan, $450, Pending)
```

```
    BusinessRequest(103, Morgan, $3500, Pending)
```

```
    BusinessRequest(104, Riley, $89, Pending)
```

Point out explicitly: Taylor's request (101) doesn't appear in the "Pending requests" list — correctly excluded, since it was approved on the line above — but it's still counted in "Total requests: 4" and the "\$5,239.00" total, which include every request regardless of status. Ask the room: "why does `total_amount()` include the approved request, but `list_pending()` doesn't show it?" (Because they're answering different questions — `total_amount()` is a sum over *everything* in the collection; `list_pending()` is explicitly filtered to one status. Neither is "more correct" — they're deliberately different views over the same underlying data.)

Common student mistakes to watch for:

- Confusing `mgr.requests[0]` (the first *added* request) with "the request with `request_id == 101`" — these happen to coincide in this example (Taylor was both added first and has ID 101), but they're conceptually different: list position versus a data field's value. Worth a brief note that a genuinely robust "find by ID" method isn't built today, only positional access — a good forward-looking observation if a student asks how they'd approve a *specific* request by ID rather than by list position.
- Calling `BusinessRequest(...)` and forgetting to pass it into `add_request(...)` — creates an object that's immediately discarded (no variable holds onto it, and it was never added to the manager), silently vanishing with no error; a good "the report shows fewer requests than expected" debugging scenario if it happens.

Check for understanding: "If you called `mgr.add_request(BusinessRequest(105, 'Alex', 'Travel', 2000))` right after this script's current last line, then re-ran `mgr.summary_report()`, would the earlier report's Taylor-approved status still be reflected?" (Yes — `mgr` and every request already added to it persist for as long as the script keeps running; adding a fifth request doesn't reset or affect the earlier four in any way. A good check that students see the manager's list as accumulating state over the whole script's run, not resetting between operations.)

6.4 Part 3 — OOP to SQL Mapping (0:30–0:42, 12 min)

Teaching goal: Explicitly map today’s OOP vocabulary to SQL vocabulary — genuine groundwork for Weeks 13–14’s database content, not a tangential exercise.

Say to the class:

“This table looks simple, but I want you to take it seriously — in three weeks, you’re building real SQL databases, and every single row of this table is a concept you already understand today, just under a different name. Getting comfortable with the translation now makes Week 13 dramatically easier.”

Do, live, filling in README.md:

OOP to SQL Mapping

OOP concept	SQL equivalent	This system
-----	-----	-----
class BusinessRequest	table: requests	[your table name]
instance (one object)	row (one record)	[example row]
attribute: amount	column: amount REAL	[your type]
list_pending()	SELECT WHERE status=?	[your query]
total_amount()	SELECT SUM(amount)	[your aggregate]

Line-by-line explanation, filling in a model “This system” column live:

- `class BusinessRequest` → `table: requests` — say explicitly: a class definition is a *blueprint* describing what fields every instance will have; a SQL table definition (`CREATE TABLE requests (...)`) is the same idea — a fixed set of named, typed columns every row will have.
- `instance (one object)` → `row (one record)` — `req_101` from Week 10 corresponds to one specific row in a `requests` table — e.g., `(101, 'Taylor', 'Travel', 1200, 'Approved')`. Model this explicitly: “This system” column might read something like `(101, 'Taylor', 'Travel', 1200.00, 'Approved')`.
- `attribute: amount` → `column: amount REAL` — an object’s attribute and a table’s column play the same role: one named piece of data, one type, present on every instance/row. `REAL` is SQLite’s floating-point type name (worth a brief forward-reference: Week 14 covers SQLite specifically) — worth noting the *type* also needs mapping, not just the name: Python’s `float` and SQL’s `REAL` (or `FLOAT/DOUBLE` in other database systems) are the same underlying idea, different vocabulary.
- `list_pending()` → `SELECT WHERE status=?` — say explicitly, this is the mapping worth dwelling on most: **a method that filters a Python list is doing conceptually the same job as a SQL WHERE clause filtering table rows**. `list_pending()`’s `[r for r in self.requests if r.status == 'Pending']` and `SELECT * FROM requests WHERE status`

= 'Pending' are answering the *identical* question, in two different languages/paradigms.

- `total_amount()` → `SELECT SUM(amount)` — same idea: `sum(r.amount for r in self.requests)` and `SELECT SUM(amount) FROM requests` both aggregate a numeric column/attribute across every row/instance.

Common student mistakes to watch for:

- Filling in the “This system” column with vague or generic answers instead of genuinely specific ones drawn from their own `models.py` — push for actual field names and actual example values, not placeholder text; this table is meant to be a real, useful reference students can look back at in Week 13, not a completed-for-completion’s-sake formality.
- Treating this table as disconnected from today’s actual code — worth explicitly connecting each row back to a specific line already written in Part 1: “which line of `RequestManager` does `list_pending()`’s SQL row correspond to?”

Check for understanding: “If you needed to add a new business rule — say, requests over \$5,000 need a ‘Director’ approval tier — what would change in the Python class, and what would change in a hypothetical SQL version?” (In Python: likely a new method, or an update to an existing one, checking the amount against a new threshold. In SQL: likely a new `WHERE` clause or `CASE` expression in a query, not a change to the table structure itself, unless a new column were needed to store the tier explicitly. A good, forward-looking question that previews the kind of thinking Week 13’s database design will require.)

6.5 Part 4 — Six pytest Tests (0:42–0:58, 16 min)

Teaching goal: Six new tests covering the composed system — RequestManager holding and filtering BusinessRequest objects — including, again, a dedicated independence test, now for *managers* rather than individual requests.

Say to the class:

“Six new tests, added to last week’s file — keep your Week 10 tests, don’t delete them. And notice: the very last test is independence again, but one level up — not ‘are two requests independent,’ but ‘are two managers independent.’”

Live-code this, added to tests/test_models.py:

```
from models import BusinessRequest, RequestManager

def test_manager_starts_empty():
    mgr = RequestManager()
    assert len(mgr.requests) == 0

def test_add_increases_count():
    mgr = RequestManager()
    mgr.add_request(BusinessRequest(1, 'A', 'Travel', 500))
    assert len(mgr.requests) == 1

def test_list_pending_filters_correctly():
    mgr = RequestManager()
    req1 = BusinessRequest(1, 'A', 'Travel', 500)
    req2 = BusinessRequest(2, 'B', 'Travel', 600)
    mgr.add_request(req1); mgr.add_request(req2)
    req1.approve()
    pending = mgr.list_pending()
    assert len(pending) == 1
    assert pending[0].requester == 'B'

def test_total_amount_correct():
    mgr = RequestManager()
    mgr.add_request(BusinessRequest(1, 'A', 'T', 500))
    mgr.add_request(BusinessRequest(2, 'B', 'T', 750))
    assert mgr.total_amount() == 1250

def test_get_by_status():
    mgr = RequestManager()
```

```

    req1 = BusinessRequest(1, 'A', 'T', 500)
    mgr.add_request(req1)
    req1.approve()
    approved = mgr.get_by_status('Approved')
    assert len(approved) == 1

def test_independent_managers():
    mgr1 = RequestManager()
    mgr2 = RequestManager()
    mgr1.add_request(BusinessRequest(1, 'A', 'T', 500))
    assert len(mgr2.requests) == 0

```

Line-by-line explanation:

- `test_manager_starts_empty` — confirms `__init__`'s `self.requests = []` — a simple but genuinely worthwhile check: this is the *foundation* every other manager test depends on being true.
- `test_list_pending_filters_correctly` — **the richest test here, worth walking slowly:** two requests are added, `req1` is approved, and the test asserts *two* things — the pending list has exactly one item (`len(pending) == 1`), **and** that remaining item is specifically `req2`, not just “some request” (`pending[0].requester == 'B'`). Say explicitly why the second assertion matters: a version of `list_pending()` with a subtly wrong filter condition might still happen to return a list of length 1 by coincidence, on this specific test data — checking *which* request survived the filter is a stronger, more specific verification than just checking the count.
- `test_independent_managers` — **the composition-level independence test**, directly parallel to Week 10's `test_instances_are_independent`, now one level up: two separate `RequestManager` objects, a request added to only one, and confirming the other's `self.requests` remains empty. Say explicitly: this confirms `RequestManager` objects are independent of *each other* in exactly the same way `BusinessRequest` objects were shown to be independent last week — the same underlying Python behavior (every object has its own separate attributes), now demonstrated at a different level of the design.

Run it:

```
pytest -v
```

Verified output — all tests (Week 10's seven plus today's six) pass:

```

tests/test_models.py::test_default_status_is_pending PASSED
tests/test_models.py::test_approve_changes_status PASSED
tests/test_models.py::test_reject_changes_status PASSED
tests/test_models.py::test_requires_review_over_limit PASSED
tests/test_models.py::test_requires_review_under_limit PASSED

```

```
tests/test_models.py::test_requires_review_at_boundary PASSED
tests/test_models.py::test_instances_are_independent PASSED
tests/test_models.py::test_manager_starts_empty PASSED
tests/test_models.py::test_add_increases_count PASSED
tests/test_models.py::test_list_pending_filters_correctly PASSED
tests/test_models.py::test_total_amount_correct PASSED
tests/test_models.py::test_get_by_status PASSED
tests/test_models.py::test_independent_managers PASSED
13 passed
```

Common student mistakes to watch for:

- Accidentally deleting or overwriting Week 10's tests while adding today's — the lab page's own explicit instruction is "keep your Week 10 tests"; a quick scroll through `test_models.py` before running confirms all thirteen are present.
- In `test_list_pending_filters_correctly`, checking `pending[0].requester == 'A'` instead of `'B'` — a natural mix-up worth catching: `req1` (requester `'A'`) was the one *approved*, so it's `req2` (requester `'B'`) that remains pending and should be the sole item in the filtered list.

Check for understanding: "How is `test_independent_managers` similar to, and different from, Week 10's `test_instances_are_independent`?" (Similar: both prove that acting on one object doesn't affect a separate object of the same class. Different: Week 10's test operated on two `BusinessRequests` directly; today's operates on two `RequestManagers`, checking that each manager's own *held collection* — not a simple attribute like `status` — is independently tracked. Getting a student to articulate this parallel-but-not-identical relationship confirms genuine understanding of composition's implications, not just pattern-matching the test's shape.)

6.6 Part 5 — Ritual and Push (0:58–1:06, 8 min)

Teaching goal: The established ritual, now verifying thirteen accumulated tests across two weeks of OOP content.

Say to the class:

“Same ritual — and notice `pytest -v` now confirms thirteen tests, not six or seven — everything from last week is still being verified alongside today’s new content, every single run.”

Live-code this:

```
ruff format . && ruff check . && pytest -v  
git add . && git commit -m 'lab 11: OOP II RequestManager composition' && git push
```

Common student mistakes to watch for: None new; the main thing worth confirming is that the growing test suite (thirteen tests now, likely more in coming weeks) still runs quickly and cleanly as a single `pytest -v` call — worth stating explicitly that this is a genuine, practical benefit of the testing discipline built since Week 4: the suite scales without extra effort on the developer’s part.

Check for understanding: “If a future week’s change to `BusinessRequest` accidentally broke `requires_review()`, would today’s `test_list_pending_filters_correctly` test catch that?” (No — that test doesn’t call `requires_review()` at all; it only exercises status-based filtering. A good reminder that a large, passing test suite doesn’t mean *everything* is verified — only the specific behaviors each individual test actually checks.)

6.7 Stretch — Light Inheritance (1:06–1:15, as time allows)

Frame as a genuine preview of Week 12’s deeper inheritance content, worth real attention if the room reaches it:

```
class TravelRequest(BusinessRequest):
    def __init__(self, request_id, requester, amount, destination):
        super().__init__(request_id, requester, 'Travel', amount)
        self.destination = destination

    def requires_review(self):
        return self.amount > 500
```

Line-by-line explanation, if you demo this live:

- `class TravelRequest(BusinessRequest):` — the parentheses after the class name specify a **parent class**: `TravelRequest` is a `BusinessRequest`, inheriting everything `BusinessRequest` already has (`approve()`, `reject()`, `__repr__`, etc.) automatically, without redefining any of it.
- `super().__init__(request_id, requester, 'Travel', amount)` — calls the **parent class’s** `__init__` directly, say explicitly: this runs `BusinessRequest`’s original constructor logic, setting `request_id`, `requester`, `category` (hardcoded here to `'Travel'`, since every `TravelRequest` is, by definition, that category), `amount`, and the default `status = 'Pending'` — all without retyping any of those four lines from Week 10.
- `self.destination = destination` — a genuinely **new** attribute, specific to `TravelRequest`, that plain `BusinessRequest` objects don’t have at all.
- `def requires_review(self):` — **this method name already exists on the parent class** — redefining it here **overrides** the inherited version specifically for `TravelRequest` objects: any `TravelRequest`’s `.requires_review()` call uses *this* \$500 threshold, not `BusinessRequest`’s original \$1,000 one, while ordinary `BusinessRequest` objects are completely unaffected and keep using \$1,000.

Verified behavior:

```
t = TravelRequest(201, 'Alex', 600, 'NYC')
print(t.requires_review())    # True -- $600 > $500, TravelRequest's own threshold
print(t.category)             # Travel -- set automatically via super().__init__
print(t.destination)          # NYC -- new attribute, not on BusinessRequest at all
```

One sentence of framing worth stating even briefly: “This is inheritance in miniature — reuse what already works, override only what’s genuinely different for this specific kind of request. Week 12 goes much deeper into when this is the right design choice versus composition, which is what today’s `RequestManager` used instead.”

7 Wrap-Up (last ~9 minutes)

Review the submission checklist together:

- ☐ Git commit made, with a message including “lab 11”
- ☐ `models.py` contains both `BusinessRequest` (Week 10) and `RequestManager` (today), unmodified from each other
- ☐ `main.py` creates a manager, adds four requests, approves one, and prints the report
- ☐ `README.md` contains the completed OOP-to-SQL mapping table, with genuine “This system” answers
- ☐ `tests/test_models.py` contains all thirteen tests (seven from Week 10, six new), all passing
- ☐ Full ritual run (format, lint, test, commit, push)
- ☐ GitHub repository URL pasted into Canvas

Preview Week 12: “Today’s Stretch — `TravelRequest` inheriting from `BusinessRequest` — becomes the main event next week: OOP design decisions, when to choose inheritance versus composition, and enough hands-on practice with both to be genuinely comfortable before Unit 4’s database work begins.”

8 Appendix A — Full Answer Key (`models.py` additions + `main.py` + `README.md` section + `tests/test_models.py` additions)

```
# models.py -- added below Week 10's BusinessRequest class

class RequestManager:
    """Manages a collection of business requests."""

    def __init__(self):
        self.requests = []

    def add_request(self, request):
        """Add a BusinessRequest to the collection."""
        self.requests.append(request)

    def list_pending(self):
        """Return all requests with status Pending."""
        return [r for r in self.requests if r.status == 'Pending']

    def total_amount(self):
        """Return the total value of all requests."""
        return sum(r.amount for r in self.requests)
```

```

def get_by_status(self, status):
    """Return requests matching the given status."""
    return [r for r in self.requests if r.status == status]

def summary_report(self):
    """Print a formatted summary of all requests by status."""
    pending = self.list_pending()
    approved = self.get_by_status('Approved')
    print(f'Total requests: {len(self.requests)} | Total: ${self.total_amount():,.2f}')
    print(f'Pending: {len(pending)} | Approved: {len(approved)}')
    print('Pending requests:')
    for r in pending:
        print(f' {r}')

```

```

# main.py
from models import BusinessRequest, RequestManager

mgr = RequestManager()
mgr.add_request(BusinessRequest(101, 'Taylor', 'Travel', 1200))
mgr.add_request(BusinessRequest(102, 'Jordan', 'Equipment', 450))
mgr.add_request(BusinessRequest(103, 'Morgan', 'Software', 3500))
mgr.add_request(BusinessRequest(104, 'Riley', 'Travel', 89))

mgr.requests[0].approve()
mgr.summary_report()

```

OOP to SQL Mapping

OOP concept	SQL equivalent	This system
class BusinessRequest	table: requests	requests(id, requester, category, amount,
instance (one object)	row (one record)	(101, 'Taylor', 'Travel', 1200.00, 'Approv
attribute: amount	column: amount REAL	amount REAL NOT NULL
list_pending()	SELECT WHERE status=?	SELECT * FROM requests WHERE status = 'Per
total_amount()	SELECT SUM(amount)	SELECT SUM(amount) FROM requests

```

# tests/test_models.py -- added below Week 10's tests
from models import BusinessRequest, RequestManager

def test_manager_starts_empty():

```

```

mgr = RequestManager()
assert len(mgr.requests) == 0

def test_add_increases_count():
    mgr = RequestManager()
    mgr.add_request(BusinessRequest(1, 'A', 'Travel', 500))
    assert len(mgr.requests) == 1

def test_list_pending_filters_correctly():
    mgr = RequestManager()
    req1 = BusinessRequest(1, 'A', 'Travel', 500)
    req2 = BusinessRequest(2, 'B', 'Travel', 600)
    mgr.add_request(req1); mgr.add_request(req2)
    req1.approve()
    pending = mgr.list_pending()
    assert len(pending) == 1
    assert pending[0].requester == 'B'

def test_total_amount_correct():
    mgr = RequestManager()
    mgr.add_request(BusinessRequest(1, 'A', 'T', 500))
    mgr.add_request(BusinessRequest(2, 'B', 'T', 750))
    assert mgr.total_amount() == 1250

def test_get_by_status():
    mgr = RequestManager()
    req1 = BusinessRequest(1, 'A', 'T', 500)
    mgr.add_request(req1)
    req1.approve()
    approved = mgr.get_by_status('Approved')
    assert len(approved) == 1

def test_independent_managers():
    mgr1 = RequestManager()
    mgr2 = RequestManager()
    mgr1.add_request(BusinessRequest(1, 'A', 'T', 500))
    assert len(mgr2.requests) == 0

```

Stretch (TravelRequest inheritance):

```
class TravelRequest(BusinessRequest):
    def __init__(self, request_id, requester, amount, destination):
        super().__init__(request_id, requester, 'Travel', amount)
        self.destination = destination

    def requires_review(self):
        return self.amount > 500
```

9 Appendix B — Extra Practice (only if the class finishes early)

Four required parts plus the ritual fill the full class period at a normal pace. If a section moves unusually fast:

Extra — a `remove_request` method. Have students add `def remove_request(self, request_id): self.requests = [r for r in self.requests if r.request_id != request_id]` to `RequestManager`, and write a test confirming the collection shrinks by exactly one when a matching ID is removed, and stays the same length if a non-existent ID is passed. Good extra rehearsal of the list-comprehension-as-filter pattern, now used to *exclude* rather than *include*.

Extra — a `count_by_category` method. Have students add a method returning a dictionary mapping each category to a count of requests in that category (the accumulate-into-a-dict pattern from Week 6's Extra Practice, now reading `r.category` instead of a dictionary key), and write one test confirming a known category count on a small, hand-built manager.